1. Problem Title & Link

Title: LeetCode 283 : Move Zeroes

Link: LeetCode 283 - Move Zeroes

2. Problem Statement (Short Summary)

You are given an integer array nums.

Move all 0s to the end of the array while maintaining the relative order of non-zero elements.

You must do this **in-place**, meaning modify the existing array without creating another array.

In simple terms:

- Keep all non-zero elements in the same order
- Push all zeros to the end
- Do not use extra space

3. Examples (Input → Output)

Example 1

Input:

nums=[0,1,0,3,12]

Output:

[1,3,12,0,0]

Explanation:

Move all zeros to the end while preserving order.

Example 2

Input:

nums=[0]

Output:

[0]

Example 3

Input:

nums=[1,2,3]

Output:

[1,2,3]

Explanation:

No zeros exist.

4. Constraints

1 <= nums.length <= 10^4

−2^31 <= nums[i] <= 2^31−1

Special restrictions:

- Must modify array in-place
- Minimize operations

5. Core Concept (Pattern / Topic)

Two Pointers

6. Thought Process (Step-by-Step Explanation)

Brute Force

Create a new array:

1. Store all non-zero values
2. Count zeros
3. Add zeros at the end

Time Complexity:

O(n)

Space Complexity:

O(n)

Not acceptable because problem requires in-place solution.

Optimized Thinking

Use two pointers:

- i → scans the array
- insertPos → position where next non-zero should go

Idea:

Whenever a non-zero element appears:

- Swap it with insertPos
- Move insertPos

This automatically shifts all non-zero values to the front while moving zeros behind.

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[0,1,0,3,12]

Process:

i

0 1 0 3 12
↑

insertPos=0

---

1 found

Swap:

1 0 0 3 12
↑
insertPos=1

---

3 found

1 3 0 0 12
  ↑
insertPos=2

---

12 found

1 3 12 0 0
    ↑
insertPos=3
Final:
[1,3,12,0,0]

8. Pseudocode (Language Independent)
insertPos=0

```
for i from 0 to n-1:
   if nums[i]!=0:
      swap(nums[i],nums[insertPos])
      insertPos++
return nums
```

9. Code Implementation

Python

```
class Solution:
   def moveZeroes(self, nums):
      insertPos = 0
      for i in range(len(nums)):
         if nums[i] != 0:
            nums[insertPos], nums[i] = nums[i], nums[insertPos]
            insertPos += 1
```

Java

```
class Solution {
   public void moveZeroes(int[] nums) {
      int insertPos = 0;
      for(int i=0;i<nums.length;i++){
         if(nums[i]!=0){
            int temp=nums[i];
```

```
            nums[i]=nums[insertPos];
            nums[insertPos]=temp;

            insertPos++;
        }
    }
  }
}
```

10. Time & Space Complexity

Time Complexity:

O(n)

Reason:

- Single traversal of array

Space Complexity:

O(1)

Reason:

- Uses only pointer variables

11. Common Mistakes / Edge Cases

1. Creating a new array unnecessarily
2. Forgetting in-place modification requirement
3. Losing order of non-zero elements

Wrong:

[1,12,3,0,0]

Correct:

[1,3,12,0,0]

4. Missing cases:

[0,0,0]

[1,2,3]

[0]

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[0,1,0,3,12]

i	nums[i]	insertPos	Array State

0	0	0	[0,1,0,3,12]
1	1	0	[1,0,0,3,12]
2	0	1	[1,0,0,3,12]
3	3	1	[1,3,0,0,12]
4	12	2	[1,3,12,0,0]

Final Output:

[1,3,12,0,0]

13. Common Use Cases (Real-Life / Interview)

Data processing:

- Move invalid values to the end

Streaming systems:

- Filter useful records first

Database operations:

- Push null values to end

Memory optimization:

- Compact active elements

14. Common Traps (Important!)

1. Using extra arrays
2. Forgetting to increment pointer
3. Swapping incorrectly
4. Breaking relative order
5. Using nested loops unnecessarily

15. Builds To (Related LeetCode Problems)

Progression:

- LC 283 → Move Zeroes
- LC 26 → Remove Duplicates from Sorted Array
- LC 27 → Remove Element
- LC 88 → Merge Sorted Array
- LC 75 → Sort Colors

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes

Brute Force Extra Array	O(n)	O(n)	Simple but extra memory
Shift Elements Repeatedly	$O(n^2)$	O(1)	Too slow
Two Pointers	O(n)	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

The algorithm always keeps insertPos at the next valid position for a non-zero element. As non-zero elements are encountered, they are moved forward while preserving order, causing zeros to naturally move toward the end.

18. Variations / Follow-Up Questions

1. Move all negative numbers to the end
2. Move all even numbers to the front
3. Partition array based on a condition
4. Solve for linked list instead of array
5. Minimize total swaps further
6.